Lecture 25.08: Naive Bayes Implementation in Scikit-Learn

📌 Topic Overview

In this lecture, Shraddha ma'am demonstrates the Python implementation of **GaussianNB** using Scikit-Learn, training the model on continuous features and inspecting the resulting accuracy and classification report.

🛠️ Scikit-Learn Workflow

1. Split raw data into `X_train`, `X_test`, `y_train`, `y_test`.
2. Initialize `GaussianNB()`.
3. Fit the model using `model.fit(X_train, y_train)`.
4. Predict test samples with `model.predict(X_test)`.
5. Evaluate using `accuracy_score` and `classification_report`.

💻 Python Implementation

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

def train_and_evaluate_gaussian_nb():
    """
    Trains a GaussianNB classifier on a classification dataset,
    evaluates its performance, and returns metrics.
    """
    # 1. Synthetic continuous feature dataset
    X, y = make_classification(
        n_samples=500, n_features=6, n_informative=4, random_state=42
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # 2. Fit Gaussian Naive Bayes
    gnb = GaussianNB()
```

```
    gnb.fit(X_train, y_train)

    # 3. Evaluate
    y_pred = gnb.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred)

    return gnb, acc, report

# --- Function Call & Testing ---
model, accuracy, metrics_report = train_and_evaluate_gaussian_nb()
print(f"GaussianNB Accuracy: {accuracy*100:.1f}%")
print("Classification Report:")
print(metrics_report)
```

🔑 Key Takeaways

1. `GaussianNB` requires zero tuning parameters; it runs instantaneously because fitting simply calculates feature means and variances.
2. It serves as an exceptional fast baseline model for any classification benchmark.